HW5 — Neural-network verification

CS 3892 / CS 5892 · Formal Methods: Safe and Trustworthy AI · Fall 2026

Out	Tuesday, December 1
Due	**Thursday, December 10, 11:59 p.m. US Central**
Weight	4% of the course grade (100 points)
Tools	auto_LiRPA (IBP, CROWN, α-CROWN), an exact MILP encoding solved by HiGHS, a PGD attack; VNN-LIB
Zero-install path	the HW5 notebook in Google Colab
Submit	Brightspace → Assignments → *HW5 — Neural network verification*

Start here: github.com/ttj/cs3892-examples has the starters in `homework/hw5-nn-verification/`, and the HW5 notebook installs the tools and runs everything in Colab. Each starter runs as it is, with a placeholder where your work goes.

What you need, and when you get it.

- **Tuesday Dec 1** covers **Parts 1, 2 and 4**: robustness as a formal query, VNN-LIB, IBP and CROWN bounds, complete vs incomplete.
- **Thursday Dec 3** covers **Parts 3 and 5**: tools and VNN-COMP, what "verified" and "timeout" mean, and LLM-written specifications.

This is deliberately the lightest homework: the final projects land the same week.

What this is for

The network is the AI component now. A verifier can prove that a network's answer does not change anywhere in a small box around an input. Two things decide what such a proof is worth:

- **which verifier** answered: a sound but incomplete one can say "unknown" where a complete one decides;
- **which property** it was given.

The second is D7's question again. A property that means something other than what was intended is still verified or falsified, with full confidence, about the wrong thing.

The network, the tools, VNN-LIB

The network:

- **Inputs:** MNIST digits downsampled to 14×14, so 196 pixels, each in [0, 1]. They are `X_0` … `X_195`, row by row.
- **Architecture:** 196 → 32 → ReLU → 32 → ReLU → 10.
- **Outputs:** the logits `Y_0` … `Y_9`; the predicted class is the largest.
- **Test images:** ten, where image *i* is a digit *i*.

`hw5lib.py` holds the network, the images and three tools. Read it, but do not edit it: the checker refuses a changed copy.

tool		answers
`lirpa_check(W, spec, method)`	auto_LiRPA's bounds, `"IBP"`, `"CROWN"` or `"alpha-CROWN"`: **sound, incomplete**	verified · unknown
`milp_check(W, spec)`	the network as a mixed-integer linear program (one 0/1 variable per ReLU that can be both on and off), solved by HiGHS: **sound and complete**	verified · falsified (with a counterexample) · timeout
`pgd_attack(W, spec)`	projected gradient descent	falsified · no counterexample found

VNN-LIB, the format of the VNN-COMP benchmarks, is a subset of SMT-LIB:

```
(declare-const X_0 Real)            ; ... one per input and one per output
(assert (>= X_0 0.0))               ; the input box: a lower and an upper bound on every
X_i
(assert (<= X_0 0.02))
(assert (or (and (>= Y_3 Y_4))      ; the output condition
            (and (>= Y_5 Y_4))))
```

A VNN-LIB file states the counterexample. Its assertions describe the inputs in the box together with the outputs **where the property fails**. A verifier answers "verified" (`unsat`) when no input in the box reaches that output region, and "falsified" (`sat`) when it finds one. Writing down the property you want, instead of its violation, is the classic mistake.

`hw5lib.parse_vnnlib` reads the subset used here:

- `declare-const`, and `assert` with `<=`, `>=`, `<`, `>`;
- `and` and `or`;
- `+`, `-`, multiplication by a constant, and numbers.

`<` and `>` are read as `<=` and `>=`; no property here depends on the boundary.

Running the tools

In Colab. Open the HW5 notebook and run it top to bottom. It installs the pinned tools (about a minute, once per runtime) and runs every starter. Edit the files with the file browser (folder icon, left), save, and re-run. **A Colab runtime is deleted when it disconnects: download your files before you leave.**

On your machine, or in a Codespace, preferably in a virtual environment:

```
git clone --depth 1 https://github.com/ttj/cs3892-examples.git
cd cs3892-examples/homework/hw5-nn-verification
python3 -m pip install -r requirements.txt
python3 -m pip install --no-deps --ignore-requires-python -r requirements-lirpa.txt
python3 check_hw5.py
```

Every version is pinned: torch 2.11.0 (CPU), auto_LiRPA 0.8.0, highspy 1.15.1 and numpy 2.2.6. `check_hw5.py` reports any difference. It checks Parts 1, 2, 3 and 5 and reports each as *ok*, *placeholder*, *wrong* (with the reason) or *error*. `python3 check_hw5.py --strict` exits 0 only when every part is *ok*.

Part 1 — Robustness as a formal query *(20 points)*

1. Write the property *"every input within L∞ distance ε of the image x_0, with every pixel kept in [0, 1], is classified as its label y"* as a formula over x and the network f. *(4)*
2. Write `make_vnnlib(x0, label, eps)` in `part1_query.py`. It returns the VNN-LIB text for that property. The checker compares its **meaning** (box pixel by pixel, unsafe output region as a set) with the property on five queries. *(8)*
3. Run `python3 part1_query.py` and report the three tools' verdicts for image 4 at ε = 0.02, 0.03 and 0.05. Explain why CROWN's "unknown" at 0.03 is not a "no", and what the MILP adds. *(4)*
4. The last line re-runs the MILP on ε = 0.03 with a 0.01 s time limit. What does its verdict let you conclude about the property? *(4)*

Part 2 — The ε frontier *(20 points)*

1. Write `binary_search(pred, lo, hi, tol)` in `part2_frontier.py`, and state the invariant your loop keeps. *(4)*
2. Run `python3 part2_frontier.py`. For images 0 and 9 it finds:
   - **ε_C:** the largest ε CROWN verifies;
   - **a bracket on the exact robustness radius r*:** from the MILP;
   - **ε_A:** the smallest ε at which PGD finds a counterexample.

   Report the numbers. Explain why ε_C ≤ r* ≤ ε_A must hold for **any** sound verifier and any attack. *(8)*
3. What lives between ε_C and r*, and between r* and ε_A? Why is incompleteness, not a bug, the usual reason for a gap? *(4)*
4. Binary search needs a monotone predicate. "The MILP verifies at ε" is monotone; why? Is "CROWN verifies at ε" guaranteed to be? And "PGD finds nothing at ε"? What does your search return if a predicate is not monotone? *(4)*

Part 3 — Two verifiers on the same queries *(20 points)*

1. Run `python3 part3_compare.py`: CROWN and the MILP on 30 queries (10 images × 3 values of ε). Of the queries where CROWN says "unknown", how many does the MILP verify, and how many does it falsify? *(6)*
2. Set `FINDING` to one query where CROWN says "unknown" and the MILP decides. Explain what CROWN approximates that the MILP does not, and why the MILP's way does not scale. *(6)*
3. Compare the times. Which queries are slow for the MILP, and why those? *(4)*
4. The script stops with an error if CROWN ever verifies a query the MILP falsifies. Why must that never happen, and what would it mean if it did? *(4)*

Part 4 — IBP, CROWN, α-CROWN: precision against cost *(20 points)*

`python3 part4_bounds.py` prints, for five queries, each method's sound lower bound on the worst margin min_{j≠y} (Y_y − Y_j) over the box, which is positive exactly when that method verifies, and the time taken.

1. Report the table. *(8)*
2. IBP verifies none of these queries. Explain why interval bounds lose so much here, and connect it to what a non-relational domain lost at a join in session 18. Why are CROWN's linear bounds tighter? *(6)*
3. Find the query where α-CROWN verifies and CROWN does not. What does α-CROWN optimise, what does it cost, and when is it worth it? *(6)*

Part 5 — AI component: VNN-LIB from English *(20 points)*

`part5/properties.md` states three properties in English: robustness, a target class that must never be on top, and a margin. Give an LLM the three sentences, the network's interface, and the pixel values of the images. You may also ask it for a program that writes the files. Ask it for one VNN-LIB file per property.

1. Save its answers verbatim as `part5/P5a_llm.vnnlib`, `P5b_llm.vnnlib` and `P5c_llm.vnnlib`, and your prompt in `part5/prompt.md`. Run `check_hw5.py`. For each file, report what it says:
   - a syntax error;
   - a match;
   - a difference in meaning: in the input box, or in the output region, with the output vector on which the file and the property disagree.

   Say what the LLM did wrong. *(8)*
2. Write corrected versions, `part5/P5a_fixed.vnnlib` and the other two. The checker must report a match for each. *(6)*
3. Run CROWN and the MILP on the LLM's files and on yours, and report every verdict. If a wrong property was **verified**, explain what that verification is worth. *(6)*

What to submit

One archive or repository link containing:

1. Your `part1_query.py`, `part2_frontier.py` and `part3_compare.py`, and the six `.vnnlib` files and `prompt.md` in `part5/`. `python3 check_hw5.py --strict` is the acceptance test.
2. A short report, Markdown or PDF, about two pages. It answers every numbered question, with the tool output it relies on.
3. The AI-use disclosure (below).

Late policy

Four late days for the semester, in whole-day units, no more than two on any one homework, no questions asked. Beyond that, 20% per day.

Collaboration and AI use

Discussion is encouraged; submitted work must be your own. **AI assistants are allowed and expected**, and Part 5 is about using one. Two conditions:

1. **Disclose.** Say what you used and for what.
2. **Verify.** You are responsible for everything you submit. Here the checker and the verifiers are the ground truth. Notice which of them can tell you that a property **means** what you intended: only one, and only because the intended property was written down independently.